

Pandas e NumPy – Curso básico

Universidade Estadual da Paraíba - UEPB



1 [Pandas](#)

2 [Numpy](#)

Pandas

Manipulação e transformação de dados clínicos



Afinal, o que é o Pandas?



O "Excel" do Python: É uma biblioteca construída especificamente para analisar e manipular tabelas de dados.

De onde vem o nome? Vem de *Panel Data* (Dados em Paineis), um termo da estatística. Não tem relação com o urso!

Por que não usar o Excel? O Pandas não trava. Ele processa milhões de linhas de pacientes em segundos.

Para que serve na prática? Abrir, limpar e cruzar enormes volumes de dados clínicos para preparar o terreno para a Inteligência Artificial.

Código:

Para começar a usar, basta "chamar" essa ferramenta no seu código:

```
1 import pandas as pd
```

Observação: "pd" é apenas um apelido carinhoso para não precisarmos digitar a palavra inteira toda vez.

Estruturas de dados





Series (1D) vs DataFrame (2D)

Como o Pandas organiza um sistema de saúde:

- **DataFrame (2D):** É a tabela inteira (linhas e colunas). Imagine a ficha completa com os dados de todos os pacientes.
- **Series (1D):** É uma única coluna. Imagine que você arrancou apenas a coluna "Idade" para analisá-la isoladamente.



Na prática

Um **DataFrame** é composto pela colagem de várias **Series**:

```
3 dados = pd.DataFrame ({
4     "paciente" : ["Ana", "João"],
5     "idade" : [45, 62],
6     "glicemia" : [99, 140]
7 })
```

Acessando e espiando dados



Isolando informações

Para pegar apenas a coluna de interesse:

```
1 dados.idade
```

idade	
0	45
1	62

```
1 dados['idade']
```

idade	
0	45
1	62

Observação

*Os colchetes funcionam sempre. O ponto falha se o nome da coluna tiver espaços (ex: "Pressão Arterial").

Espiando os prontuários

Você não quer imprimir 10 mil linhas de uma vez.

```
1 dados.head() #5 primeiras linhas  
2 dados.tail() #5 últimas linhas  
3 dados.sample() #Sorteia 1 paciente
```

	paciente	idade	glicemia
0	Ana	45	99
1	João	62	140
2	Maria	38	87
3	Carlos	55	115
4	Fernanda	29	92

Figura: Output de dados.head()

	paciente	idade	glicemia
2	Maria	38	87
3	Carlos	55	115
4	Fernanda	29	92
5	Lucas	71	158
6	Beatriz	48	105

Figura: Output de dados.tail()

	paciente	idade	glicemia
2	Maria	38	87

Figura: Output de dados.sample()

Compreendendo a Estrutura (Raio-X)



Formato da tabela

Entendendo o tamanho da sua pesquisa clínica:

```
1 dados.columns
2 #Mostra o nome de todas as colunas
3
4 dados.shape
5 #Mostra(linhas,colunas)
6 #Ex: (5000,15)
```

```
Index(['paciente', 'idade', 'glicemia'], dtype='object')
```

Figura: Output de dados.columns

```
(7, 3)
```

Figura: Output de dados.shape

Estatísticas rápidas

```
1 dados.info()
2 #Diz os tipos de dados (texto, numero)
3 #e se há dados faltantes
4
5 dados.describe()
6 #Calcula automaticamente a média,
7 #mínimo e máximo de todos os números
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  -
0  paciente  7 non-null         object
1  idade     7 non-null         int64
2  glicemia  7 non-null         int64
dtypes: int64(2), object(1)
memory usage: 300.0+ bytes
```

Figura: Output de dados.info()

	idade	glicemia
count	7.000000	7.000000
mean	49.714286	113.714286
std	14.279522	26.240845
min	29.000000	87.000000
25%	41.500000	95.500000
50%	48.000000	105.000000
75%	58.500000	127.500000
max	71.000000	158.000000

Figura: Output de dados.describe()

Valores únicos



☰ .unique()

Quais são os valores distintos de uma coluna? (ex:

Quais os tipos sanguíneos cadastrados?).

```
1 dados.sangue.unique()
```

```
array(['A+', 'O+', 'B+', 'AB+', 'O-', 'A-', 'B-'])
```

Figura: Output de dados.sangue.unique()

.nunique()

Conta **quantos** valores diferentes existem, em vez de listá-los um a um.

```
1 dados.sangue.nunique()
```

```
7
```

Figura: Output de dados.sangue.nunique()

Identificando problemas nos dados



Repetições no sistema

Pacientes cadastrados em duplicidade enviam
qualquer modelo de IA.

```
1 dados.duplicated()
2 #Marca quais linhas são repetidas
3
4 dados.drop_duplicates()
5 #Remove e limpa a tabela
```

0	paciente	idade	glicemia	tipo_sanguineo
0 False	0 Ana	45	99	A+
1 False	1 João	62	140	O+
2 False	2 Maria	38	87	B+
3 True	4 Carlos	55	115	AB+
4 False	5 Fernanda	29	92	O-
5 False	6 Lucas	71	158	A-
6 False	7 Beatriz	48	105	B-
7 False				

Figura: Output de dados.drop_duplicates()

Figura: Output de dados.duplicated()

Desbalanceamento

Verificando a frequência de cada categoria.

```
1 dados['cidade'].value_counts()
2 #Mostra quantos pacientes existem
3 #de cada cidade na sua base
```

	count
cidade	
Recife	3
Campina Grande	2
João Pessoa	1
Natal	1
Fortaleza	1

Figura: Output de dados['cidade'].value_counts()

O pesadelo: dados ausentes



```

1 dados.isnull().sum()
2 #Conta quantos estão faltando
3
4 dados.dropna()
5 #Remove o paciente com o dado faltante
6
7 dados.fillna(0)
8 #Preenche o buraco com zero
9
10 dados.replace(-1, np.nan)
11 #Troca erro de digitação (-1) por NaN

```

- Na saúde, é normal faltar preencher algum dado. O Pandas marca esses buracos como **np.nan** (Não é um número). Eles quebram a IA.

	0					
paciente	0					
idade	0					
glicemia	1					
sangue	0					
cidade	0					

	paciente	idade	glicemia	sangue	cidade
0	Ana	45	99.0	A+	Campina Grande
1	João	62	140.0	O+	João Pessoa
3	Maria	38	87.0	B+	Recife
4	Carlos	55	115.0	AB+	Natal
5	Fernanda	29	-1.0	O-	Fortaleza
6	Lucas	71	158.0	A-	Recife
7	Beatriz	48	105.0	B-	Campina Grande

Figura: Output de dados.dropna()

	paciente	idade	glicemia	sangue	cidade
0	Ana	45	99.0	A+	Campina Grande
1	João	62	140.0	O+	João Pessoa
2	Maria	38	0.0	B+	Recife
3	Maria	38	87.0	B+	Recife
4	Carlos	55	115.0	AB+	Natal
5	Fernanda	29	-1.0	O-	Fortaleza
6	Lucas	71	158.0	A-	Recife
7	Beatriz	48	105.0	B-	Campina Grande

Figura: Output de dados.fillna(0)

	paciente	idade	glicemia	sangue	cidade
0	Ana	45	99.0	A+	Campina Grande
1	João	62	140.0	O+	João Pessoa
2	Maria	38	NaN	B+	Recife
3	Maria	38	87.0	B+	Recife
4	Carlos	55	115.0	AB+	Natal
5	Fernanda	29	NaN	O-	Fortaleza
6	Lucas	71	158.0	A-	Recife
7	Beatriz	48	105.0	B-	Campina Grande

Figura: Output de dados.replace(-1, np.nan)

Figura: Output de dados.isnull().sum()

Conversão de tipos



Ajustando a rota

```
1 dados['idade'] = \
2 dados['idade'].astype(int)
3
4 #Converte a coluna para números
5 #inteiros (int) definitivamente|
```

- Às vezes, uma idade como 20 é importada do Excel pelo sistema como se fosse um texto "20".

Se você tentar calcular a média de idades com textos, o Python vai acusar um erro.

- O comando `.astype()` garante a conversão do tipo da coluna.
- **Obrigatório:** Fazer isso antes de tentar plotar gráficos ou enviar para modelos matemáticos de Machine Learning.

Formatando e Transformando



Remove espaços invisíveis no início e final de textos digitados no sistema.

Código Python

```
# Remove espaços antes e depois dos nomes  
df['paciente'] = df['paciente'].str.strip()
```

ANTES DO STRIP:

```
[' Ana Lima ', 'Carlos Souza ', ' Beatriz Santos ']
```

DEPOIS DO STRIP:

```
['Ana Lima', 'Carlos Souza', 'Beatriz Santos']
```

Troca valores específicos em uma coluna para padronizar cadastros clínicos (ex: transformar 'Masc' e 'M' em 'Masculino').

Código Python

```
# Padronizando termos
df['sexo'] = df['sexo'].replace({
    'F': 'Feminino',
    'M': 'Masculino',
    'Masc': 'Masculino'
})
```

Antes

... Antes:

	paciente	sexo
0	Ana Lima	F
1	Carlos Souza	Masc
2	Beatriz Santos	Feminino
3	Lucas Rocha	M

Depois

... Depois:

	paciente	sexo
0	Ana Lima	Feminino
1	Carlos Souza	Masculino
2	Beatriz Santos	Feminino
3	Lucas Rocha	Masculino

Ajusta o número de casas decimais de colunas numéricas, facilitando a leitura de exames laboratoriais e medições clínicas.

Código Python

```
# Arredondando glicemia (1 decimal) e creatinina (2 decimais)
df['glicemia'] = df['glicemia'].round(1)
df['creatinina'] = df['creatinina'].round(2)
```

Antes

...	Antes			
		paciente	glicemia	creatinina
0	Ana Lima	99.4582	0.8542	
1	Carlos Souza	140.1234	1.2981	
2	Beatriz Santos	88.7891	0.7123	

Depois

...	Depois			
		paciente	glicemia	creatinina
0	Ana Lima	99.5	0.85	
1	Carlos Souza	140.1	1.30	
2	Beatriz Santos	88.8	0.71	

Converte texto (object) em formato de data real (datetime64), permitindo operações e filtros temporais. Coisa que em String não é possível ser feita.

Código Python

```
# Converte a coluna de texto para formato Data
df['data_admissao'] = pd.to_datetime(df['data_admissao'])
```

Antes

```
... data_admissao    object
dtype: object
Antes:
```

	paciente	data_admissao
0	Ana Lima	2023-03-15
1	Carlos Souza	2024-07-20
2	Beatriz Santos	2024-09-02

Depois

```
... data_admissao    datetime64[ns]
dtype: object
Depois:
```

	paciente	data_admissao
0	Ana Lima	2023-03-15
1	Carlos Souza	2024-07-20
2	Beatriz Santos	2024-09-02

Utiliza o acessor `.dt` para isolar componentes da data, como o ano de atendimento ou internação do paciente.

Código Python

```
# Extraindo o ano
df['ano_admissao'] = df['data_admissao'].dt.year
df[['paciente', 'data_admissao', 'ano_admissao']]
```

...	paciente	data_admissao	ano_admissao
0	Ana Lima	2023-03-15	2023
1	Carlos Souza	2024-07-20	2024
2	Beatriz Santos	2024-09-02	2024

Filtra linhas que contenham um termo específico em textos clínicos (ex: localizar pacientes diagnosticados com 'diabetes', ignorando maiúsculas e minúsculas).

Código Python

```
# Filtra diagnósticos contendo 'diabetes'
filtro = df['diagnostico'].str.contains('diabetes', case=False)
df_diabetes = df[filtro]
```

Antes (Base completa)

... Base Original:

	paciente	diagnostico
0	Ana Lima	Diabetes tipo 2
1	Carlos Souza	Hipertensão Arterial
2	Beatriz Santos	Pré-diabetes
3	Lucas Rocha	Gripe Comum

Depois (Apenas diabetes)

... Pacientes Filtrados (com Diabetes):

	paciente	diagnostico
0	Ana Lima	Diabetes tipo 2
2	Beatriz Santos	Pré-diabetes



Ordenação



Reorganiza as linhas com base em uma ou mais colunas. Um exemplo de uso seria para priorizar atendimentos por gravidade de exames ou idade de risco.

Código Python

```
# Ordena da maior para a menor glicemia (decrecente)
df_ordenado = df.sort_values(by='glicemia', ascending=False)
```

Antes (Ordem de Chegada)

... Antes:

	paciente	idade	glicemia
0	Ana Lima	45	99.5
1	Carlos Souza	62	140.1
2	Beatriz Santos	38	88.8
3	Lucas Rocha	71	185.0

Depois (Maior Risco Primeiro)

... Depois:

	paciente	idade	glicemia
3	Lucas Rocha	71	185.0
1	Carlos Souza	62	140.1
0	Ana Lima	45	99.5
2	Beatriz Santos	38	88.8

Manipulação de índices



Define uma coluna existente (como o ID do paciente) como identificador principal das linhas e permite retornar ao índice numérico padrão quando necessário.

Código Python

```
# Define o identificador unico como indice
df_idx = df.set_index('id_paciente')

# Restaura o indice padrao numerico (0, 1, 2...)
df_reset = df_idx.reset_index()
```

`.set_index('id_paciente')`

... Com id_paciente no índice (`.set_index`):

	nome	setor
id_paciente		
101	Ana Lima	UTI
102	Carlos Souza	Triagem
103	Beatriz Santos	Enfermaria

`.reset_index()`

... Índice restaurado (`.reset_index`):

	id_paciente	nome	setor
0	101	Ana Lima	UTI
1	102	Carlos Souza	Triagem
2	103	Beatriz Santos	Enfermaria

Ao definir a data como índice (DatetimeIndex), o Pandas facilita a consulta de intervalos e dias específicos diretamente pelo `.loc[]`.

Código Python

```
# 1. Define a data como indice temporal
df_tempo = df.set_index('data')

# 2. Consulta direta das internacoes daquele dia
resultado = df_tempo.loc['2024-05-02']
```

Base Indexada por Data

*** Série Temporal (Índice = DatetimeIndex):

internacoes_diarias	
data	
2024-05-01	12
2024-05-02	18
2024-05-03	15
2024-05-04	22

Filtro Direto: `.loc['2024-05-02']`

*** Internações em 02/05/2024:

2024-05-02	
internacoes_diarias	18

Agrupamento e Agregação



Agrupa os dados com base em uma coluna de categorias (diagnósticos) e calcula resumos estatísticos, como a média de idade dos pacientes por condição clínica.

Código Python

```
# Calcula a media de idade para cada diagnostico
resumo = df.groupby('diagnostico')['idade'].mean().round(1)
resumo = resumo.reset_index()
```

Antes (Registros Individuais)

*** Base de Pacientes:

	paciente	diagnostico	idade
0	Ana	Diabetes	62
1	Carlos	Hipertensão	55
2	Beatriz	Diabetes	48
3	Lucas	Gripe	23
4	Marina	Hipertensão	67
5	João	Diabetes	71

Depois (Média por Diagnóstico)

*** Média de Idade por Diagnóstico:

	diagnostico	idade
0	Diabetes	60.3
1	Gripe	23.0
2	Hipertensão	61.0

Criando novas variáveis



Cria ou atualiza uma nova coluna com base em regras lógicas, como classificar a glicemia em 'Normal' ou 'Alterada' segundo critérios clínicos.

Código Python

```
# Define valor padrao e aplica condicao clinica
df['status'] = 'Normal'
df.loc[df['glicemia'] >= 126.0, 'status'] = 'Alterada'
```

Antes

...	Base Original:		
	paciente	glicemia	
0	Ana Lima	95.0	
1	Carlos Souza	145.0	
2	Beatriz Santos	88.0	
3	Lucas Rocha	180.0	

Depois (Status Classificado)

...	Com Coluna Condicional:			
	paciente	glicemia	status	
0	Ana Lima	95.0	Normal	
1	Carlos Souza	145.0	Alterada	
2	Beatriz Santos	88.0	Normal	
3	Lucas Rocha	180.0	Alterada	

Cria novas colunas aplicando funções modulares sobre colunas existentes de forma limpa e encadeada.

Código Python

```
# Funcao para calculo do IMC
def calcular_imc(dados):
    return (dados['peso'] / (dados['altura']**2)).round(1)

# Adiciona a nova coluna com .assign()
df = df.assign(imc=calcular_imc)
```

Antes (Peso e Altura)

... Antes do Cálculo:

	paciente	peso	altura
0	Ana Lima	68.0	1.65
1	Carlos Souza	85.0	1.75
2	Beatriz Santos	54.0	1.60

Depois (IMC Calculado)

Depois (.assign):

	paciente	peso	altura	imc
0	Ana Lima	68.0	1.65	25.0
1	Carlos Souza	85.0	1.75	27.8
2	Beatriz Santos	54.0	1.60	21.1

Removendo Colunas



Elimina colunas desnecessárias ou variáveis sensíveis da base de dados usando o parâmetro `axis=1` para indicar a remoção no eixo das colunas.

Código Python

```
# Remove a coluna 'observacoes_internas'
df_limpo = df.drop('observacoes_internas', axis=1)
```

Antes

Base Original (com `observacoes_internas`):

	paciente	diagnostico	observacoes_internas
0	Ana Lima	Diabetes	Anotação 1
1	Carlos Souza	Hipertensão	Anotação 2
2	Beatriz Santos	Diabetes	Anotação 3

Depois (Coluna Removida)

Base Limpa (sem a coluna):

	paciente	diagnostico
0	Ana Lima	Diabetes
1	Carlos Souza	Hipertensão
2	Beatriz Santos	Diabetes

Salvando Resultados



Permite exportar os DataFrames tratados para arquivos físicos em formatos amplamente utilizados para compartilhamento e relatórios.

Código Python

```
# Exportando para CSV
df_final.to_csv('relatorio_pacientes.csv', index=False, encoding='utf-8-sig')

# Exportando para planilha Excel nativa (.xlsx)
df_final.to_excel('relatorio_pacientes.xlsx', index=False, sheet_name='Triagem')
```

- **`.to_csv()`**: Cria um arquivo de texto separado por vírgulas.
- **`.to_excel()`**: Gera uma planilha Excel formatada, permitindo definir o nome da aba via `sheet_name`.
- **`index=False`**: Evita criar uma coluna indesejada com os índices numéricos (0, 1, 2...) no arquivo final.
- **`encoding='utf-8-sig'`**: Garante a exibição correta de acentos e caracteres especiais no Excel em português.

Numpy



Introdução ao NumPy

O que é?

É a principal biblioteca de computação numérica e matemática do Python.

```
1 import numpy as np
```

- Ele trabalha com Arrays, que são estruturas de alta performance (feitas na linguagem C, que é muito mais rápida que o Python).

Porque importar se temos Pandas?

Se o Pandas é o organizador de tabelas, o NumPy é o motor matemático dele.

- O Pandas foi construído em cima do NumPy.
- Por trás das cortinas, cada coluna do Pandas guarda seus números dentro de um Array do NumPy para conseguir processar os cálculos tão rápido.

Arrays vs Listas

- A **Lista do Python** é como uma maleta médica genérica: cabe gaze, seringa, esparadrapo (texto, número, booleanos). É flexível, mas muito lenta para a IA procurar e calcular coisas.
- O **Array do NumPy** é como uma bandeja cirúrgica: só aceita um tipo de coisa (ex: apenas números). Essa rigidez permite que o computador processe milhares de operações de uma vez só (Vetorização).

```
1 lista = [1,2,3]
2 arr = np.array ([1,2,3])
3
4 #Acessando e cortando (slicing)
5 arr[0] #Retorna 1
6 arr[0:2] #Retorna [1,2]
```

Arrays Especiais

np.arange()

Gera uma sequência numérica espaçada, como uma régua.

```
1 np.arange (0,10,2)
2 # [0 2 4 6 8]
```

```
array([0, 2, 4, 6, 8])
```

Figura: Output de np.arange()

np.zeros()

Cria matriz preenchida com zeros. Muito usado para inicializar pesos em IA.

```
1 np.zeros(3)
2 # [0., 0., 0.]
```

```
array([0., 0., 0.])
```

Figura: Output de np.zeros()

np.ones()

Semelhante ao zeros, mas preenche estruturas matemáticas com o número 1.

```
1 np.ones ((2,2))
```

```
array([[1., 1.],
       [1., 1.]])
```

Figura: Output de np.ones()

Matrizes multidimensionais e IA

■ Um array não precisa ser apenas uma linha de dados (1D). Ele pode formar tabelas e cubos.

```
1 m = np.array ([[1,2,3],  
2               [4,5,6]])  
3  
4 m.shape #Dimensões: [2,3]  
5 m.ndim #Quantidade dimensões: 2
```

A Ponte: De Pixels a Modelos

■ O computador não enxerga "ossos" ou "tecidos". Uma **Radiografia Digital**, **Tomografia** ou **Mapa Raster** é lida pela Inteligência Artificial apenas como uma gigantesca **tabela de números (Matriz NumPy)**, onde cada número representa um tom de cinza de um pixel (0 a 255).

Reshape



■ array.reshape()

A função `.reshape()` permite remodelar as dimensões de uma matriz ou alterar o número de elementos em cada dimensão.

```
import numpy as np

array = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
print(array)
novoArray = array.reshape(4, 3)
print(novoArray)
```

```
[ 1  2  3  4  5  6  7  8  9 10 11 12]
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
```

É possível fazer o reshape de uma matriz desde que a matriz final seja uma matriz regular (todas as linhas precisam ter o mesmo número de elementos). Caso contrário, o código quebra.

- A quantidade de parâmetros passados na função define o número de dimensões da nova matriz
- A ordem dos parâmetros vai da estrutura maior para o elemento individual
 - ▶ O primeiro número define a divisão mais externa
 - ▶ O último define quantos elementos individuais existem em cada linha/coluna
- Exemplo: em uma matriz 2D passamos (linhas, colunas). Se for 3D, vira (blocos, linhas, colunas), e assim por diante.

Entretanto, sempre calcular manualmente a dimensão apropriada ao usar a função `.reshape()` é bem trabalhoso. Por isso, podemos usar o que é chamado de "Dimensão desconhecida"

- Ao colocar '-1' como um dos argumentos, o proprio numpy calcula automaticamente para você

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8,9,10,11,12])
newarr = arr.reshape(3,-1)
print(newarr)
```

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8,9,10,11,12])
newarr = arr.reshape(2,3,-1)
print(newarr)
```

```
[[[ 1  2]
   [ 3  4]
   [ 5  6]]

 [[ 7  8]
   [ 9 10]
   [11 12]]]
```

Além disso, se o '-1' for colocado sozinho dentro da função, ele transforma a matriz em um array de apenas uma unica dimensão.

```
import numpy as np
array = np.array([[1,2,3],[4,5,6],[7,8,9]])
novoArray = array.reshape(-1)
print(array)
print(novoArray)
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
[1 2 3 4 5 6 7 8 9]
```

Funções Universais



- As funções universais (ou ufuncs) são funções prontas oferecidas pelo NumPy para realizar operações vetorizadas nos arrays/matrizes.
- Operações vetorizadas são uma forma de executar grandes cálculos matemáticos em arrays, matrizes e/ou blocos de dados de uma só vez, sem precisar de laços de repetição.
- As ufuncs são utilizadas para fazer operações matemáticas e/ou lógicas de elemento a elemento
- As ufuncs, ao serem chamadas, sempre criam um novo array como resultado da operação.
- Para que a função seja realizada, os arrays e matrizes envolvidos precisam ter as mesmas dimensões/shapes

Sendo X e Y arrays que possuem as mesmas dimensões

OPERAÇÕES DE MATEMÁTICA BÁSICA

- **np.add(x,y)** : Soma uma matriz a outra, elemento por elemento
- **np.subtract(x,y)** : subtrai uma matriz da outra, elemento por elemento
- **np.multiply(x,y)** : Multiplica os elementos, elemento por elemento
- **np.divide(x,y)** : Divide os elementos, elemento por elemento
- **np.power(x,y)**: Eleva um número a potência da outra
- **np.mod(x,y)**: Retorna o resto da divisão, elemento por elemento
- **np.sqrt(array)**: calcula arquadrada de cada elemento

```
import numpy as np

a = np.array([4, 9, 16])
b = np.array([2, 3, 4])

print("Adição: ", np.add(a, b))
print("Subtração: ", np.subtract(a, b))
print("Multiplicação: ", np.multiply(a, b))
print("Divisão: ", np.divide(a, b))
print("Potência: ", np.power(a, b))
print("Resto da divisão: ", np.mod(a, b))
print("Raiz quadrada: ", np.sqrt(a))
```

```
Adição: [ 6 12 20]
Subtração: [ 2  6 12]
Multiplicação: [ 8 27 64]
Divisão: [2. 3. 4.]
Potência: [ 16 729 65536]
Resto da divisão: [0 0 0]
Raiz quadrada: [2. 3. 4.]
```

ARREDONDAMENTOS E VALORES ABSOLUTOS

- **np.absolute(array):** Transforma os números negativos em positivos (tira o sinal)
- **np.floor(array):** Arredonda para baixo (2.9 vira 2)
- **np.ceil(array) :** Arredonda para cima (2.1 vira 3)
- **np.round(array) :** Arredonda para o numero mais próximo (2.1 vira 2 e 2.9 vira 3)
- **np.trunc(array) :** Tira as casas decimais (2,43 vira 2)
- **np rint(array) :** Arredonda pro número mais próximo, mas mantém como float (2.3 vira 2.0)

```
import numpy as np

numeros = np.array([-2.7, 2.1, 3.8])

print("Valor absoluto:", np.absolute(numeros))
print("Arredonda para baixo (floor):", np.floor(numeros))
print("Arredonda para cima (ceil):", np.ceil(numeros))
print("Arredonda para o mais próximo (round):", np.round(numeros))
print("Trunca / remove decimais (trunc):", np.trunc(numeros))
print("Arredonda mantendo float (rint):", np.rint(numeros))
```

```
Valor absoluto: [2.7 2.1 3.8]
Arredonda para baixo (floor): [-3.  2.  3.]
Arredonda para cima (ceil): [-2.  3.  4.]
Arredonda para o mais próximo (round): [-3.  2.  4.]
Trunca / remove decimais (trunc): [-2.  2.  3.]
Arredonda mantendo float (rint): [-3.  2.  4.]
```

COMPARAÇÕES LÓGICAS ENTRE ARRAYS

- **`np.greater(x, y)`** : Verifica se o primeiro é maior que o segundo, elemento a elemento
- **`np.less(x, y)`** : Verifica se o primeiro é menor que o segundo, elemento a elemento
- **`np.equal(x, y)`** : Verifica se os dois são iguais, elemento a elemento
- **`np.not_equal(x, y)`** : Verifica se os dois são diferentes, elemento a elemento (logica invertida)
- **`np.logical_and(x, y)`** : Retorna True apenas se ambos os elementos forem verdadeiros, elemento a elemento
- **`np.logical_or(x, y)`** : Retorna True se pelo menos um dos elementos for verdadeiro, elemento a elemento

```
import numpy as np

a = np.array([5, 10, 15])
b = np.array([3, 10, 20])

print("Maior que (greater):", np.greater(a, b))
print("Menor que (less):", np.less(a, b))
print("Igual a (equal):", np.equal(a, b))
print("Diferente de (not_equal):", np.not_equal(a, b))

print("-----")

v1 = np.array([True, True, False])
v2 = np.array([True, False, False])

print("E lógico (logical_and):", np.logical_and(v1, v2))
print("OU lógico (logical_or):", np.logical_or(v1, v2))

Maior que (greater): [ True False False]
Menor que (less): [False False  True]
Igual a (equal): [False  True False]
Diferente de (not_equal): [ True False  True]
-----
E lógico (logical_and): [ True False False]
OU lógico (logical_or): [ True  True False]
```

EXPONENCIAL, LOGARITMO e TRIGONOMETRIA

- **np.exp(a)** : calcula o exponencial (numero de euler elevado a elemento por elemento)
- **np.log(a)** : calcula logaritmo natural (base e) de elemento a elemento
- **np.log10(a)** : calcula logaritmo na base 10 de elemento a elemento
- **np.log2(a)**: calcula logaritmo na base 2, elemento por elemento
- **np.sin(a)** : calcula o seno do angulo
- **np.cos(a)** : calcula o cosseno do angulo
- **np.tan(a)**: calcula a tangente do angulo

```
import numpy as np

numeros = np.array([1.0, 10.0, 100.0])
angulos = np.array([0, np.pi / 2, np.pi])

print("Exponencial (e^x):", np.exp(numeros))
print("Logaritmo natural (base e):", np.log(numeros))
print("Logaritmo base 10:", np.log10(numeros))
print("Logaritmo base 2:", np.log2(numeros))
print("Seno:", np.sin(angulos))
print("Cosseno:", np.cos(angulos))
print("Tangente:", np.tan(angulos))
```

Concatenação



CONCATENAR ARRAYS

- **np.concatenate(array)**: Junta dois ou mais arrays para formar um novo array.
- Os 2 arrays precisam ter tamanhos compatíveis

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

juncao = np.concatenate((a, b))
print("Resultado:", juncao)

Resultado: [1 2 3 4 5 6]
```

Funções de Agregação



- Funções de agregação servem basicamente para pegar um conjunto com vários dados e resumi-los em um único valor.
- Normalmente, eles retornam apenas um valor unico, como um inteiro ou um float
- No entanto, o parâmetro "axis" permite calcular por partes: axis=0 na vertical (por coluna) e axis=1 na horizontal (por linha), retornando um array com o resultado de cada uma em vez de apenas um único número.

- **np.sum(a)** : soma todos os valores do array
- **np.mean(a)** : Calcula a média aritmetica do array
- **np.min(a)** : Retorna o valor minimo do array
- **np.max(a)** : Retorna o valor maximo do array
- **np.argmin(a)** : Diz o indice onde o valor minimo do array esta guardado
- **np.argmax(a)** : Diz o indice onde o valor maximo do array esta guardado

```
import numpy as np

valores = np.array([10, 5, 40, 20, 15])

print("Soma:", np.sum(valores))
print("Média:", np.mean(valores))
print("Valor mínimo:", np.min(valores))
print("Valor máximo:", np.max(valores))
print("Índice do mínimo (argmin):", np.argmin(valores))
print("Índice do máximo (argmax):", np.argmax(valores))
```

```
Soma: 90
Média: 18.0
Valor mínimo: 5
Valor máximo: 40
Índice do mínimo (argmin): 1
Índice do máximo (argmax): 2
```

- **np.std(a)** : Calcula o desvio padrão dos dados. Um desvio padrão alto indica que os valores estão muito dispersos em relação à média, enquanto um valor baixo indica que os dados estão mais concentrados e próximos da média.
- **np.var(a)** : Calcula a variância (o desvio padrão ao quadrado). A diferença prática é que a variância dá um peso muito maior para valores que fogem do padrão (os extremos).

```
import numpy as np

notas = np.array([6.0, 7.0, 6.5, 7.5, 7.0])

print("Desvio padrão (std):", np.std(notas))
print("Variância (var):", np.var(notas))
```

```
Desvio padrão (std): 0.5099019513592784
Variância (var): 0.259999999999999995
```

Valores Ausentes



- Em NumPy existe um tipo de valor chamado "NaN" (Not a number, não é um número). Esse tipo de valor serve para representar vazios.
- Ao usar funções de agregação com um array onde um NaN está presente, o valor final é alterado e terá o NaN como resultado na maioria das operações.
- Para resolver esse problema, ao utilizar uma função de agregação com a presença de um NaN, basta escrever um "nan" antes da função.

EXEMPLOS:

- **np.nansum(a)** : Calcula a soma dos elementos, ignorando os valores NaN
- **np.nanmean(a)** : Calcula a média aritmética dos elementos, ignorando os valores NaN
- **np.nanmedian(a)** : Calcula a mediana dos elementos, ignorando os valores NaN
- **np.nanmax(a)** : Retorna o maior valor do array, ignorando os valores NaN
- E assim por diante com as funções de agregação...

```
import numpy as np

dados = np.array([10.0, np.nan, 20.0, 30.0, np.nan])

print("Soma sem NaN:", np.nansum(dados))
print("Média sem NaN:", np.nanmean(dados))
print("Mediana sem NaN:", np.nanmedian(dados))
print("Maior valor sem NaN:", np.nanmax(dados))
```

```
Soma sem NaN: 60.0
Média sem NaN: 20.0
Mediana sem NaN: 20.0
Maior valor sem NaN: 30.0
```

Também existe uma função que analisa elemento por elemento e retorna um array, onde coloca True nas posições em que tem um NaN presente. O `.isnan(array)`

```
import numpy as np

notas = np.array([7.5, np.nan, 9.0, np.nan, 4.0])
posicoes_vazias = np.isnan(notas)
print(posicoes_vazias)
```

```
[False  True False  True False]
```

Indexação e Comparação Booleana



INDEXAÇÃO E COMPARAÇÕES BOOLEANAS

É possível comparar e filtrar arrays usando operações booleanas. A comparação gera um array de True e False e a Indexação utiliza esse resultado como filtro para criar um novo array contendo apenas os valores desejados.

```
import numpy as np

idades = np.array([15, 20, 12, 22])
mascara = idades >= 18

print(idades)
print(mascara)
```

```
[15 20 12 22]
[False  True False  True]
```

Para comparar, faça uma comparação normal e será gerado um array com True ou False dependendo da condição.

```
import numpy as np

idades = np.array([15, 20, 12, 22])
filtrado = idades[idades >= 18]

print(idades)
print(filtrado)
```

```
[15 20 12 22]
[20 22]
```

Para indexar, coloque a condição entre colchetes depois do nome do array

O Pandas foi criado usando o NumPy como base, ou seja, ao utilizar o Pandas, na verdade estamos usando o NumPy com uma roupagem diferente.

Enquanto Numpy é pensando mais para dados numericos brutos e só aceita um tipo de valor no array, o Pandas aceita multiplos tipos de valores, é focado em tabelas e em leitura de arquivos.

Na realidade, muitas bibiliotecas do python usa o NumPy como base.

```
import pandas as pd
import numpy as np

dados = [10, 20, np.nan, 40]
coluna_pandas = pd.Series(dados)

array_escondido = coluna_pandas.to_numpy()

print(coluna_pandas)
print("-----")
print(array_escondido)
```

0	10.0
1	20.0
2	NaN
3	40.0

dtype: float64

[10. 20. nan 40.]

Obrigado!

